Assignment-13.4

G.Sai teja

2303A52135

Batch 41

Task-1: Refactoring Data Transformation Logic

PROMPT:

Write a Python program to double each number in a list. Add proper comments and a docstring explaining the program.

Legacy Code:

```python
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Refactored Code:

```python
#Write a Python program to double each number in a list. Add proper comments and a docstring explaini
"""
Program: Double the values in a list
Description:
This program takes a list of integers and creates a new list
where each element is multiplied by 2.
"""
# Step 1: Define the original list of numbers
values = [2, 4, 6, 8, 10]
# Step 2: Create an empty list to store doubled values
doubled = []
# Step 3: Loop through each number in the original list
for v in values:
    # Multiply each number by 2 and add it to the new list
    doubled.append(v * 2)
# Step 4: Display the final result
print(doubled)
```

OUTPUT:

```
[4, 8, 12, 16, 20]
PS C:\Users\saite\Downloads\AI ASSISTENT CODING>
```

EXPLANATION:

This program creates a list of numbers [4,8,12,16,20] and then uses a loop to go through each number one by one. For every number in the list, it multiplies the number by 2 and adds the result to a new list called doubled. After the loop finishes, the program prints the new list, which contains all the original numbers multiplied by 2.

Task-2: Improving Text Processing Code Readability

PROMPT:

Refactor the given Python code that builds a sentence using string concatenation inside a loop. Improve efficiency and readability while keeping the output unchanged.

Legacy Code:

```python

words = ["Refactoring", "with", "AI", "improves",
"quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Refactored Code:

```python
#Refactor the given Python code that builds a sentence using string concatenation inside a loop. Impr
"""
Program: Generate a sentence from a list of words
Description:
This program joins a list of words into a single sentence
in a more efficient and readable way.
"""
# List of words
words = ["Refactoring", "with", "AI", "improves", "quality"]
# Join words with a space between them
message = " ".join(words)
# Print the final sentence
print(message)
```

OUTPUT:

```
Refactoring with AI improves quality
PS C:\Users\saite\Downloads\AI ASSISTENT CODING>
```

EXPLANATION:

The original code used a loop to add each word to a string one by one, which is less efficient because strings are immutable in Python. Every time a word is added, a new string is created. The improved version uses the join() method, which combines all the words in the list into a single string with spaces in between. This method is faster, cleaner, and easier to read while producing the same output:Refactoring with AI improves quality.

Task-3: Safer Access to Configuration Data

PROMPT:

Refactor the given Python code to safely access dictionary values using a better method instead of manually checking for keys. Keep the output unchanged if the key is missing.

Legacy Code:

```python
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Refactored Code:

```python
#Refactor the given Python code to safely access dictionary values using a better method instead of m
"""
Program: Safely access configuration values
Description:
This program retrieves the 'timeout' value from a configuration
dictionary. If the key does not exist, it prints a default message.
"""
# Configuration dictionary
config = {"host": "localhost", "port": 8080}
# Safely access 'timeout' using get()
timeout = config.get("timeout")
# Check if timeout exists
if timeout is not None:
    print(timeout)
else:
    print("Default timeout used")
```

OUTPUT:

```
Default timeout used
PS C:\Users\saite\Downloads\AI ASSISTENT CODING>
```

EXPLANATION:

The original code manually checks whether the key "timeout" exists in the dictionary before accessing it. This works but makes the code longer. The improved version uses the get() method, which safely retrieves the value of a key. If the key does not exist, it returns a default value instead of causing an error. This makes the code shorter, cleaner, and safer while keeping the same output: Default timeout used.

Task-4: Refactoring Conditional Logic for Scalability

PROMPT:

Refactor the given Python code that uses multiple if-elif conditions to perform operations. Replace it with a cleaner and more scalable mapping-based approach while keeping the same output.

Legacy Code:

```python
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Refactored Code:

```python
#Refactor the given Python code that uses multiple if-elif conditions to perform operations. Replace
"""
Program: Perform mathematical operations using mapping
Description:
This program performs a mathematical operation based on
the given action using a dictionary mapping approach.
"""
# Given inputs
action = "divide"
x, y = 10, 2
# Define operations using lambda functions
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}
# Get the result safely
operation = operations.get(action)
result = operation(x, y) if operation else None
# Print result
print(result)
```

OUTPUT:

```
5.0
PS C:\Users\saite\Downloads\AI ASSISTENT CODING>
```

EXPLANATION:

The original code used multiple if-elif conditions to check which operation to perform. While it works, it becomes harder to maintain when more operations are added. The

refactored version uses a dictionary called operations that maps action names to small functions (lambda functions). This makes the code cleaner and easier to expand. If a new operation is needed, we can simply add it to the dictionary instead of adding more if-elif statements. The functionality remains the same, and the output is still 5.0.

Task-5: Simplifying Search Logic in Collections

PROMPT:

Refactor the given Python code that manually searches for an item in a list using a loop. Replace it with a more concise and readable approach while keeping the same output.

Legacy Code:

```python
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Refactored Code:

```python
#Refactor the given Python code that manually searches for an item in a list using a loop. Replace it
"""
Program: Check if an item exists in inventory
Description:
This program checks whether "eraser" is present
in the inventory list using a simpler method.
"""
# Inventory list
inventory = ["pen", "notebook", "eraser", "marker"]
# Use 'in' operator for direct membership checking
found = "eraser" in inventory
# Print result
print("Item Available" if found else "Item Not Available")
```

OUTPUT:

```
Item Available
PS C:\Users\saite\Downloads\AI ASSISTENT CODING>
```

EXPLANATION:

The original code used a loop to check each item in the list one by one and set a flag variable when the item was found. While this works, it makes the code longer. The improved version uses Python's in operator, which directly checks whether an item exists in the list. This makes the code shorter, easier to read, and more efficient while producing the same output: Item Available.